AI와 WebAssembly로 만드는 최소 개발 환경

업무 담당자와 개발자가
같은 요구사항과 실행 결과로 협업합니다

남정현
닷넷데브 (.NET Dev)

부산 닷넷 유니버스 11월 세미나
2026년 11월



코드를 거치는 단계를 건너뛸 수 있을까?

최근 AI 코딩 도구를 보며, 요청과 실행 사이의 절차를 줄이고 싶었습니다

요청을 코드로 옮기고
실행 환경에서 결과를 확인한다

이 중간 과정을 건너뛰고 바로 결과를 확인할 수는 없을까?

요구사항이 곧바로 코드가 된다면

비개발자의 요청을 개발자가 곧바로 읽고 실행할 코드로 표현해 보고 싶었습니다

“이 조건이면 이렇게 처리해 주세요”

작은 함수와 확인할 결과로 전달한다

업무 담당자와 개발자가 같은 것을 보고 이야기할 수 있을까?

컴파일러를 Copilot Skill로 만들기

M365 Copilot에 새로 들어온 Skill에 WebAssembly 컴파일러를 넣어 보기로 했습니다

작업 지침 어떤 C 함수와 테스트를 만들지 AI에게 설명

컴파일 도구 필요한 컴파일러를 Skill과 함께 준비

실행 결과 Wasm 파일과 호출 규칙, 테스트 기록을 받음

발표자 실험: M365 Copilot과 Copilot Studio

의미 있는 실험 결과가 나왔습니다

컴파일러를 Skill로 묶고 실행 결과까지 얻을 수 있다는 가능성을 확인했습니다

`module.wasm`	실제로 실행할 Wasm 파일
`contract.json`	함수의 형식과 파일을 식별하는 정보
`verification.json`	입력과 기대값, 실제 결과의 기록

이 실험이 두 역할의 협업에 어떤 의미가 있는지 공유합니다

함께 검증하는 개발 방식

업무 담당자와 개발자는 같은 입력과 결과를 보며 논의할 수 있습니다

검증 가능한 요구사항
그리고 함께 확인하는 실행 결과

시민 개발자(업무 담당자)는 규칙을 설명하고
전문 개발자는 앱에 연결하고 안전하게 운영합니다

오늘 살펴볼 내용

작은 호기심에서 시작한 Copilot Skill 실험과 두 개발자의 협업을 살펴봅니다

01

시민 개발자와 전문 개발자의 협업

서로의 역할을 이해하고 같은 기대 결과를 정함

02

AI와 Skill이 실행되는 환경

Harness, Sandbox, MXC와 Skill의 역할

03

세 번의 프롬프트와 .NET 연결

기본 운임, 할증, 계약 할인과 실행 가능한 코드

04

역할과 책임

요구사항을 쓰는 능력과 검증, 보안, 운영

시민 개발자의 역할

시민 개발자는 업무 지식을 바탕으로 자신의 일을 개선할 도구를 만듭니다

업무를 아는 사람	주문, 정산, 고객 응대의 규칙과 예외를 압니다
원하는 동작을 설명	필요한 입력과 결과를 정하고 업무에 맞는지 확인합니다
배송비 사례에서는	무게와 크기, 지역별 예외와 기대 배송비를 정합니다

전문 개발자의 역할

전문 개발자는 도구를 시스템에 연결하고 안전하게 운영할 조건을 설계합니다

시스템에 연결	기존 앱과 데이터에 연결하고 입력과 오류 처리를 정합니다
동작과 안전성 확인	같은 테스트를 실행하고 접근 권한과 자원 사용을 제한합니다
배포와 운영	승인한 버전을 배포하고 기록하며 문제가 생기면 복구합니다

AI 시대의 공동 검증

AI가 구현을 돕는 동안 두 역할은 같은 요구사항과 실행 결과를 함께 확인합니다

업무 담당자	규칙과 기대 결과를 설명
AI와 개발 도구	검토할 코드를 만들고 실제로 실행
두 역할이 함께	같은 입력과 결과로 올바른 동작을 판단

여러 배송 조건을 함께 다루기

무게와 크기, 할증과 할인 기준을 나누어 합의하면 변경을 검증할 수 있습니다

기본 운임은 어느 구간인가?
할증과 할인은 어디에 적용하는가?

세 번의 프롬프트로 기본 계산, 추가요금, 기업 계약 할인을 차례로 확장합니다

기본 운임을 정하는 네 구간

무게와 세 변의 합을 모두 만족하는 가장 낮은 요금을 선택합니다

무게	세 변의 합	기본 운임
3kg 이하	80cm 이하	4,000원
5kg 이하	100cm 이하	4,500원
10kg 이하	120cm 이하	6,000원
20kg 이하	160cm 이하	9,000원

무게와 각 변은 0보다 큰 숫자입니다. 20kg 또는 세 변 합 160cm를 넘으면 구조화된 오류를 반환합니다

무게와 크기의 경계값 확인

무게와 크기의 기준값을 각각 넘겨 보며 두 조건을 함께 검사합니다

무게 kg	세 변의 합 cm	기대하는 기본 운임
3	80	4,000원
3.01	80	4,500원
3	80.01	4,500원
20	160	9,000원
20.01	160	무게 초과 오류

추가 확인: 0 이하 입력은 오류, 세 변 합이 160cm를 넘으면 크기 초과 오류입니다

최소 개발 환경의 구성

요구사항과 실행 도구, 검증 결과를 한 흐름으로 연결합니다

요구사항

자연어와 합의한 테스트

무엇을 계산하고 어떤 입력을 거절할지 정함

에이전트

Harness와 Skill

AI가 지시를 읽고 필요한 도구를 사용

실행 환경

Sandbox와 런타임

프로그램이 접근할 파일과 사용할 자원을 제한

결과 파일

Wasm과 호출 규칙, 테스트 기록

실제로 사용할 앱에서 다시 실행해 확인

[MS1] [MS2] [R1]

업무에서 만나는 Microsoft 365 Copilot

M365 Copilot 라이선스를 할당받았다면 개발자가 아니어도 업무에 활용할 수 있습니다

사용하는 사람	M365 Copilot 라이선스를 할당받은 업무용 사용자
익숙한 업무 공간	평소 사용하는 Copilot 안에서 에이전트와 대화
업무에 맞게 구성	Agent Builder로 필요한 지침과 자료, Skill을 추가

사용 가능한 기능과 공유 범위는 사용자 권한과 조직의 설정을 따릅니다

[MS14] [MS15]

Skill은 지시와 실행 도구를 함께 담습니다

Skill에는 작업 지침과 참고 자료뿐 아니라 실행할 스크립트도 넣을 수 있습니다

SKILL.md	언제 사용할지, 어떤 순서로 작업할지 설명
참고 자료	업무 양식과 기준표, 테스트에 필요한 자료 제공
실행 스크립트	계산과 데이터 변환, 파일 생성과 결과 검사를 수행

이번 실험에서는 여기에 C 코드를 Wasm으로 만드는 컴파일러를 함께 묶었습니다

[MS16] [MS17]

Copilot Harness가 하는 일

Harness는 AI가 지시를 읽고 도구를 사용하도록 작업을 관리합니다

AI의 작업 진행

AI 모델에 지시와 필요한 자료 전달

필요한 도구와 Skill을 선택해 실행

대화와 작업의 진행 상태 관리

작업이 실행되는 환경

Studio가 제공하는 실행 환경

로컬 앱이 준비한 도구와 권한

제품마다 제공하는 프로그램과
접근 권한을 확인

[MS1] [MS2]

Sandbox와 MXC의 운영 방식

Studio는 실행 환경을 관리하고, 로컬 앱은 MXC로 환경을 구성합니다

구분	Studio의 관리형 Sandbox	로컬 앱의 MXC
누가 관리하나요?	Microsoft가 환경을 제공	앱 개발자가 환경을 구성
어떻게 사용하나요?	제품 안에서 코드와 파일 처리	SDK로 로컬 앱에 격리 실행 추가
도구와 권한	제품이 제공하는 도구와 정책 사용	앱에서 도구와 접근 정책을 선택
먼저 확인할 것	필요한 프로그램의 실행 가능 여부	파일, 네트워크와 프로세스 권한

Sandbox는 격리된 실행 환경을, MXC는 앱에 격리 실행을 더하는 기술을 가리킵니다
MXC: Microsoft Execution Containers

[MS2] [MS5] [MS6] [MS7]

Skill 실행에 필요한 조건

Skill을 실행하려면 필요한 프로그램과 권한을 먼저 확인해야 합니다

Skill에 담는 내용

SKILL.md에 적은 작업 지침

필요한 스크립트와 참고 자료

AI가 요청에 맞는 Skill을 선택

실행 환경에서 확인할 것

필요한 Python과 Node 버전

다른 프로그램을 실행할 권한

파일 저장과 실행 시간, 메모리 제한

[MS3] [MS4] [MS5] [R2]

Agent Builder에서 직접 만들어 보기

릴리스 패키지로 에이전트를 구성하고, 미리 보기에서 대화하며 실행해 봅니다

github.com/rkttu/wasm-skill/releases

1 릴리스 패키지 받기

c-wasm-portable-wcc.zip
릴리스에서 ZIP을 다운로드

2 Agent Builder 열기

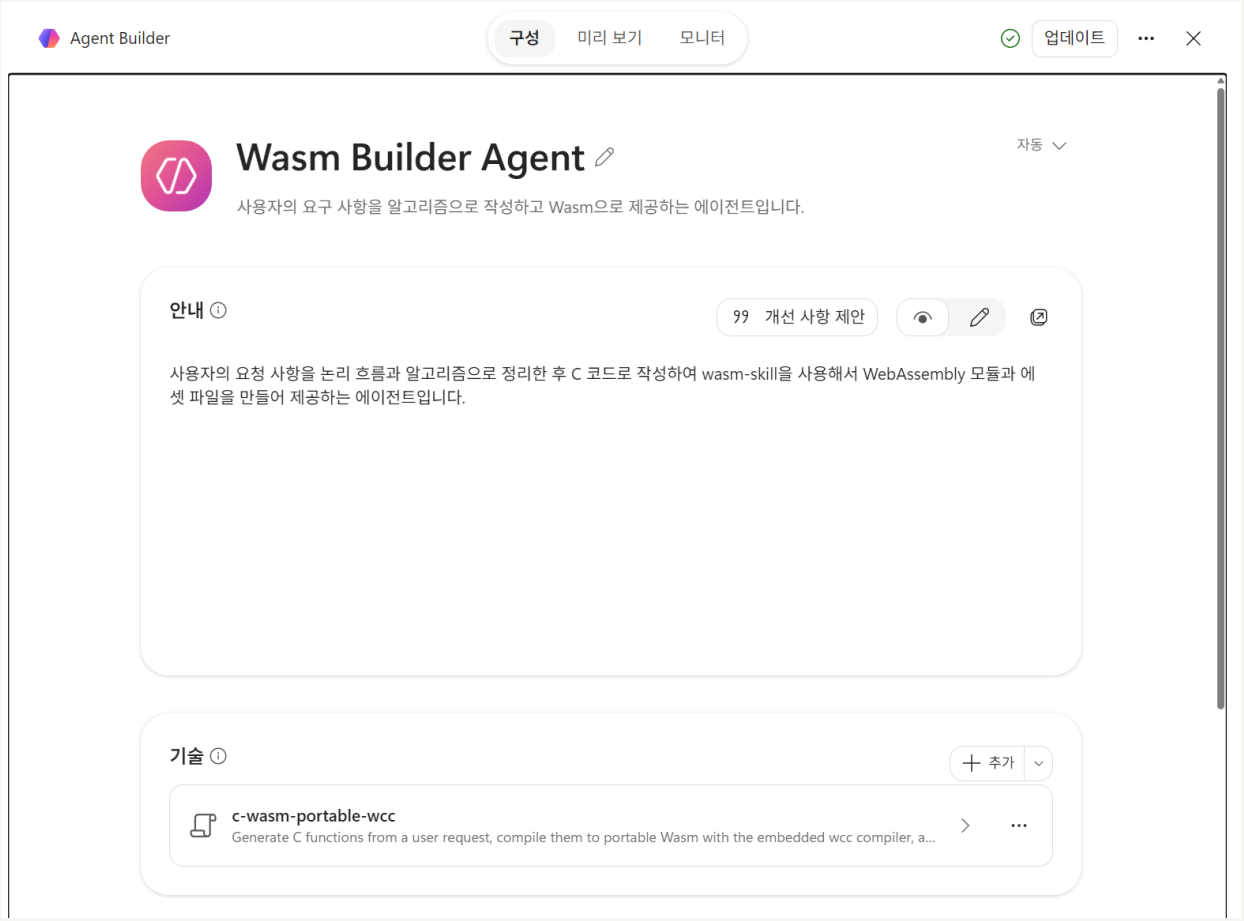
Copilot에서 에이전트 만들기
구성 화면의 '기술'로 이동

3 Skill 업로드 후 테스트

ZIP을 올리고 안내 작성
미리 보기에서 대화하며 테스트

모듈 생성 권장 모델(발표자 실험)

GPT-6.1 Sol 이상



[R7] [MS17]

문서와 실행으로 확인한 내용

공식 문서의 설명과 직접 실행한 결과를 구분해 살펴봅니다

확인 방법	확인한 내용
Microsoft 공식 문서	Harness, Skill, 관리형 실행 환경과 MXC의 역할
공개 저장소 코드	내장 컴파일러와 숫자 호출 규약, 빌드 및 검사 과정
발표자의 Copilot 실행 경험	M365 Copilot과 Copilot Studio에서 Skill 실행 새 배송비 모듈과 어댑터 생성
제공된 실제 번들 재현	JavaScript 어댑터와 Wasm의 세 단계 결과 기존 검사와 고객 할인 검사 실행

Copilot 화면과 실제 번들, 별도의 호스트 연결 예제를 각각의 근거로 설명합니다

[MS2] [R1] [R6]

C 코드가 Wasm이 되는 과정

wasm-skill은 C 코드를 컴파일하고 테스트를 통과한 결과를 저장합니다

01

AI가 C 코드와 테스트 명세를 작성

자연어 요구사항과 사람이 합의한 기대값을 사용

02

Python이 Node.js를 실행

portable 버전: Python 3.10 이상, Node.js 22 이상

03

함께 묶은 wcc 컴파일러로 변환

Node.js의 WASI에서 wcc 0.1.13 실행

04

파일을 검사하고 테스트한 뒤 저장

모든 검사를 통과해야 새 결과 폴더를 만듦

[R1] [R2] [R4] [R5]

함께 검토하는 주요 파일

계산 모듈과 호출 규칙, JSON 어댑터와 테스트 기록을 함께 봅니다

파일	함께 확인할 내용
module.c	무게와 크기, 정책을 계산하는 C 코드
module.spec.json	외부에 공개할 함수와 숫자 테스트
module.wasm	실제로 실행할 계산 모듈
contract.json	함수의 입력 형식과 파일의 해시
verification.json	모듈 빌드 과정에서의 숫자 테스트 기록
shipping.mjs	JSON 검증, 숫자 매핑과 응답 구성

업무 결과 JSON과 테스트 보고서 JSON은 각각 다른 목적의 자료입니다

[R4] [R5]

contract.json에 담긴 정보

JSON 형식과 별도로 Wasm 함수의 실제 호출 규약을 확인합니다

```
calculate_shipping
(f64, f64, f64, f64) -> i32

JSON fields:
weight_kg, width_cm,
length_cm, height_cm
```

세 버전에서 유지한 기본 함수

double 네 개를 받아 int 요금을 반환

JSON의 숫자 필드를 호스트에서 매핑

할증과 할인에는 새 정책 함수를 추가

JSON 문자열을 Wasm에 직접 전달하지 않고, 호스트 어댑터가 숫자 인자와 응답을 연결합니다

[R3] [R4]

verification.json에 담긴 결과

숫자 함수의 검증 기록과 JSON 정책 검증을 각각 확인합니다


```
{
  "export": "calculate_shipping",
  "args": [3, 30, 30, 20],
  "expected": 4000,
  "actual": 4000,
  "passed": true
}
```


실제 번들의 숫자 테스트

입력과 기대값, 실제 반환값을 비교

JSON 타입과 정책은 어댑터도 검사

회귀와 고객 할인 검사는 별도 기록

화면의 528개 집계는 별도 통합 검사 결과이며, 이 테스트 한 항목과 구분합니다

[R5]

시연 1

국내 택배 배송비를 계산하는 calculate_shipping 함수를 작성하고 WebAssembly 모듈로 빌드해주세요.

함수는 화물의 무게(kg)와 포장 상자의 가로, 세로, 높이(cm)를 입력받습니다. 세 변의 합과 무게를 각각 평가하고 두 조건을 모두 만족하는 가장 저렴한 요금 구간을 선택합니다.

요금 구간은 다음과 같습니다.
3kg 이하 및 80cm 이하: 4,000원
5kg 이하 및 100cm 이하: 4,500원
10kg 이하 및 120cm 이하: 6,000원
20kg 이하 및 160cm 이하: 9,000원

입력과 출력은 JSON으로 표현합니다. 출력에는 기본 요금, 지역 할증, 취급 할증, 할인액, 최종 요금, 적용 규칙을 각각 포함합니다. 현재 구현하지 않는 항목의 금액은 0원으로 설정합니다.

입력값이 0 이하이거나 허용 크기를 초과하면 구조화된 오류를 반환합니다. 외부 API, 네트워크, 파일 시스템에 접근하지 않는 순수 함수로 구현합니다.

이후의 기능 확장을 고려하여 입력 JSON에 추가 필드가 있더라도 기본 계산을 정상 수행하도록 설계합니다. WebAssembly 실행 환경의 실제 호출 규약을 준수하고 테스트 가능한 모듈을 생성해주세요.

시연 2

기존 calculate_shipping 함수에 새로운 배송 정책을 반영해주세요.

첫째, 배송 지역에 따라 추가 요금을 부과합니다.
MAINLAND: 추가 요금 없음
JEJU: 3,000원 추가
REMOTE: 5,000원 추가

둘째, fragile 값이 true이면 파손주의 취급 수수료 1,000원을 추가합니다.

최종 요금은 기본 요금과 각 할증 요금을 합산합니다. region을 생략하면 MAINLAND, fragile을 생략하면 false로 처리합니다. 알 수 없는 지역 코드는 오류로 처리합니다.

기존 기본 요금 규칙과 출력 JSON 구조를 변경하지 마세요. 기존 테스트를 통과하면서 추가 규칙을 검증하는 테스트를 작성하고 수정된 WebAssembly 모듈을 빌드해주세요.

시연 3

기존 calculate_shipping 함수를 기업 계약 고객도 사용할 수 있도록 확장해주세요.

고객 유형을 나타내는 customerType을 새롭게 지원합니다.
PERSONAL: 할인 없음
BUSINESS: 기본 운임의 20% 할인

할인은 기본 운임에만 적용합니다. 제주 및 도서산간 할증과 파손주의 취급 수수료에는 할인을 적용하지 않습니다. 할인액에서 원 미만의 금액은 버립니다.

최종 계산식은 다음과 같습니다.
최종 요금 = 기본 운임 - 계약 할인 + 지역 할증 + 취급 할증

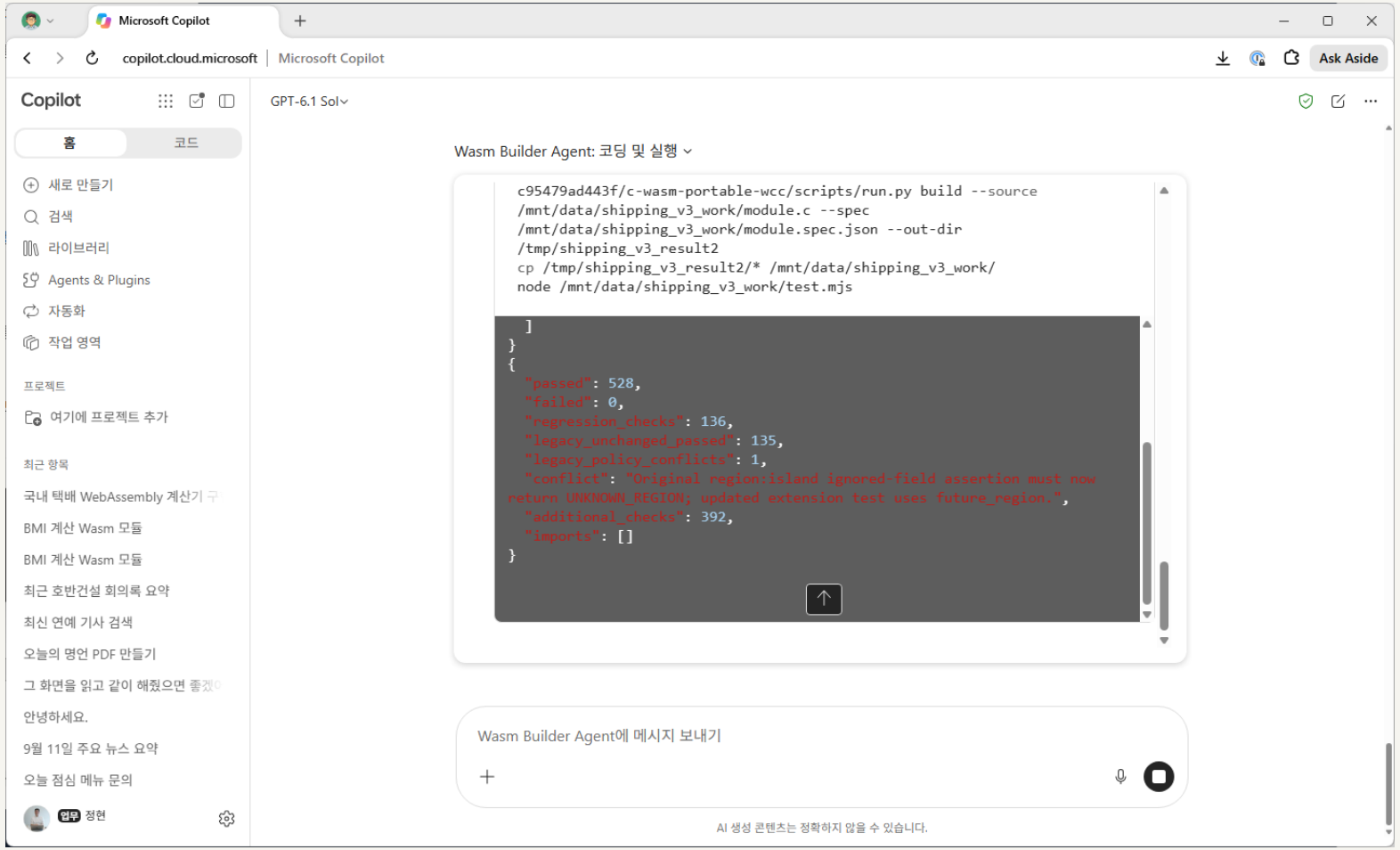
또한 반환하는 JSON의 appliedRules 배열에 적용된 요금 구간, 지역 할증, 취급 수수료, 계약 할인의 규칙 식별자를 순서대로 기록합니다.

customerType을 생략하면 PERSONAL로 처리하고 알 수 없는 유형은 오류로 처리합니다.

이전 버전의 입력 및 출력 계약을 유지하고 기존 테스트가 모두 통과하는지 검증해주세요. 새 계약 할인 규칙을 테스트한 후 WebAssembly 모듈을 다시 빌드해주세요.

M365 Copilot으로 Wasm 모듈 만들기

GPT-6.1 Sol과 컴파일러 Skill로 두 개발자가 함께 검토할 결과물을 얻었습니다



자연어로 요청

업무 규칙과 기대값을
설명합니다

모델과 도구로 실행

GPT-6.1 Sol
+ Wasm 컴파일러 Skill

같은 결과를 검토

모듈과 호출 규칙,
검증 기록을 함께 확인

세 프롬프트로 달라진 계산 결과

같은 기본 입력에 새 정책을 적용해 4,000원, 8,000원, 7,200원을 확인합니다

요금 항목 (단위: 원)	v1 기본	v2 할증	v3 할인
기본 운임	4,000	4,000	4,000
지역 할증	0	3,000	3,000
취급 할증	0	1,000	1,000
계약 할인 (차감)	0	0	−800
최종 요금	4,000	8,000	7,200

공통 입력: 3kg, 30×30×20cm
v2에 JEJU와 fragile:true, v3에 customerType:BUSINESS를 추가한 실제 결과입니다

정책 확장과 호환성 확인

새 필드에 의미를 부여하면 이전의 무시 규칙과 충돌하는 사례도 검토합니다

상황	함께 판단할 내용
v1의 추가 입력	아직 모르는 region:island는 무시
v2의 새 지역 정책	region이 정식 필드가 되어 UNKNOWN_REGION 반환
기존 검사 136개	135개 유지, 정책 충돌 1건을 별도 기록
확장성 검사	아직 정의하지 않은 future_region 필드로 확인

“모든 테스트가 통과했다”와 “모든 정책의 의미가 그대로다”를 나누어 확인합니다

JSON을 .NET과 Wasm에 연결하기

호스트가 JSON을 숫자로 바꾸고 Wasm의 반환값으로 응답을 만듭니다

JSON을 다루는 C# 어댑터

필수 필드와 숫자 타입을 검사

지역, 취급, 고객 유형을 숫자로 매핑

공통 JSON 응답과 규칙 배열을 구성

계산을 맡는 Wasm 모듈

기존 기본 함수는 f64 네 개 입력

v2와 v3의 정책 함수로 최종 계산

해시와 함수 형식, 실행 한도를 확인

[MS9] [MS10] [W1]

연결 해설 버전별 함수 선택

기존 기본 함수는 남겨 두고 v2와 v3에 새 정책 함수를 추가합니다

JSON 검증과 응답 구성은 호스트가 담당

```
v1  calculate_shipping(double × 4)

v2  calculate_shipping_with_policy(double × 4, int × 2)

v3  calculate_shipping_with_customer(double × 4, int × 3)
```

확인: 3kg, 30×30×20cm에서 기본 함수는 세 버전 모두 4,000원을 반환하나요?

[MS9] [W1]

연결 해설 1 JSON을 읽는 C# 파일 준비

C# 파일 하나에서 Wasmtime을 참조해 Wasm 모듈과 JSON 입력을 읽습니다

```
#:package Wasmtime@48.0.2

using System.Security.Cryptography;
using System.Text.Json;
using Wasmtime;

var directory = Path.GetFullPath(args[0]);
var wasmBytes = File.ReadAllBytes(
    Path.Combine(directory, "module.wasm"));
using var input = JsonDocument.Parse(
    File.ReadAllText(args[1]));
```

```
dotnet run --file app.cs -- shipping_v3_bundle business-input.json
```

[MS11] [MS12] [W1]

연결 해설 2 Wasm 실행 환경 준비

확인한 모듈을 메모리와 연산 한도 안에서 실행합니다

```
using var config = new Config().WithFuelConsumption(true);
using var engine = new Engine(config);
using var module = Module.FromBytes(engine, "shipping", wasmBytes);

if (module.Imports.Count != 0)
    throw new InvalidDataException("Unexpected Wasm imports");

using var linker = new Linker(engine);
using var store = new Store(engine);
store.SetLimits(16 * 1024 * 1024);
store.Fuel = 100_000;

var instance = linker.Instantiate(store, module);
```

contract.json의 해시와 같은 모듈인지 먼저 확인하고, 요청마다 연산 예산을 설정합니다

[MS11] [MS12] [W1]

연결 해설 3 JSON 값을 실제 함수에 연결

지역과 고객 유형을 정수로 바꿔 실제 Wasm 함수를 호출합니다

```
var regionId = region switch {
    "MAINLAND" => 0, "JEJU" => 1, "REMOTE" => 2, _ => -1 };
var customerId = customer switch {
    "PERSONAL" => 0, "BUSINESS" => 1, _ => -1 };

// Validate the input and enum ids before calling Wasm.
var calculate = instance.GetFunction<
    double, double, double, double, int, int, int, int>(
    "calculate_shipping_with_customer")!;
var finalFee = calculate(
    values[0], values[1], values[2], values[3],
    regionId, fragile ? 1 : 0, customerId);
```

앞의 double 네 개는 무게와 세 변이며, 마지막 int는 반환값입니다.

[MS11] [MS12] [W1]

연결 해설 4 계산 결과를 JSON으로 반환

호스트가 반환값을 요금 항목과 적용 규칙이 담긴 JSON 응답으로 구성합니다

```
var response = CalculateJson(
    File.ReadAllText(args[1]));
var json = JsonSerializer.Serialize(response, options);

Console.WriteLine(json);

// v3: 3 kg, 30 x 30 x 20 cm, JEJU, fragile, BUSINESS
// base_fee: 4000, regional_surcharge: 3000
// handling_surcharge: 1000, discount_amount: 800
// final_fee: 7200
// applied_rules and appliedRules explain the result.
```

실제 .NET 결과: 기본 4,000원 / 제주와 파손주의 8,000원 / 기업 계약 할인 7,200원

[MS11] [MS12] [W1]

시민 개발자에게 필요한 글쓰기

업무 지식을 조건과 예외, 기대 결과가 분명한 글로 표현합니다

역량	새 배송비 사례에서 하는 일
자연어	무게와 크기, 지역과 고객의 업무 의도를 설명
논리적 글쓰기	기본 운임 할인과 추가요금의 적용 범위를 명확히 표현
테크니컬 라이팅	JSON 필드, 기본값, 오류와 호출 방법을 기록
프롬프트 엔지니어링	세 단계의 작업과 유지할 규칙, 테스트를 요청
요구사항 공학	기대 결과와 경계값을 합의하고 변경을 검토

명확하게 쓴 요구사항이 구현의 출발점이 되고, 실행 결과로 계속 확인합니다

전문 개발자가 책임질 일

전문 개발자는 결과를 시스템에 연결하고 검증과 보안, 운영을 책임집니다

책임	구체적으로 할 일
통합	JSON 타입과 기본값을 검사하고 숫자 호출 규약에 연결
검증	경계값, 정책, 기존 응답과 추가 필드의 의미를 확인
보안	외부 기능과 파일, 네트워크 접근 및 자원 사용을 제한
운영	모듈의 해시와 버전, 승인과 배포, 이전 버전 복구를 관리

Wasm과 테스트 결과를 근거로 검토하고, 운영 주체가 최종 사용을 결정합니다

[MS8] [R3] [W1]

Wasm이 잘 맞는 일과 활용 가능성

작고 독립적인 계산에 적합하며, 다른 언어와의 연결도 검토할 수 있습니다

이런 계산에 잘 맞습니다

작고 독립적인 계산 규칙

검증 가능한 입력과 출력 계약

계산 모듈과 JSON 연결 계층을 분리

다른 언어로도 확장할 수 있습니다

.NET: Wasmtime 연결 라이브러리

Python: wasmtime / Java: Chicory

Java와 Python 연동은 이번 시연 범위 밖이며
각 환경에서 추가 검증이 필요합니다

단순 설정은 JSON, 복잡한 데이터와 외부 연결은 API나 규칙 엔진도 비교합니다

[R3] [W1] [W3] [W4]

함께 확인하는 개발 순서

규칙의 합의부터 구현과 검토, 운영까지 같은 기준을 이어갑니다

합의

업무 규칙과 기대하는 결과를 정함

모호한 조건과 예외를 먼저 합의

생성

AI가 검토할 코드를 작성

실제 도구가 컴파일과 테스트를 수행

검토

호출 규칙과 실행 결과를 함께 확인

업무 담당자는 규칙을, 개발자는 실행 조건을 검토

통합

승인한 모듈을 사용할 앱에 연결

버전, 배포, 실행 기록과 이전 버전 복구를 관리

같은 요구사항과 실행 결과로 나누는 대화

검증 가능한 요구사항과 같은 실행 결과가 두 역할의 대화를 연결합니다

시민 개발자와 전문 개발자가
같은 요구사항과 실행 결과를 봅니다

AI가 구현을 돕고, Wasm 파일로 같은 기능을 전달합니다
두 역할은 같은 결과를 바탕으로 올바른 동작을 논의합니다

질문과 의견

여러분의 업무에서도 조건과 기대 결과가 분명한 규칙부터 시작할 수 있습니다

여러분의 업무에서는
어떤 규칙을 함께 검증하겠습니까?

공식 문서와 저장소 링크는
각 슬라이드의 발표자 노트에서 확인할 수 있습니다

배송비 예제 직접 살펴보기

세 번의 프롬프트로 만든 모듈과 .NET 실행 예제를 GitHub에서 확인할 수 있습니다

wasm-shipping-bundle

생성한 Wasm 모듈과 호출 규칙
JSON 어댑터와 검증 자료
.NET 10 파일 기반 실행 예제

https://github.com/rkttu/wasm-shipping-bundle

QR 코드로 저장소를 열어 보세요

저와 링크드인에서 만나요

발표 후에도 LinkedIn에서 질문과 경험을 나눌 수 있습니다

남정현

닷넷데브 (.NET Dev)

https://linkedin.com/in/rkttu



카메라로 QR 코드를 스캔해 주세요

닷넷데브 포럼에서 이야기 나누기

.NET 개발 경험과 질문을 커뮤니티에서 함께 나눠 주세요

닷넷데브 포럼

발표 후의 질문과 경험을
포럼에서 이어가 주세요

https://forum.dotnetdev.kr



카메라로 QR 코드를 스캔해 주세요

Thank you

rkttu@rkttu.com